

GABRIEL POPA

Senior Software Engineer – React • AWS

Cluj, Romania | +40 764309991 | popagabriel0523@gmail.com | linkedin.com/in/gabriel-popa-89ba643a4

ABOUT ME

Senior Software Engineer with **11+ years of experience** building scalable **React (v16–v18)** frontends and reliable **Node.js (v12–v20)** backend APIs for modern web platforms, turning complex and evolving requirements into stable production releases. Strong expertise in **TypeScript**, **REST/GraphQL APIs**, **PostgreSQL**, **Redis**, **Kafka**, **WebSockets**, and **AWS**, with hands-on experience designing maintainable services and resilient client applications. Known for modernizing legacy systems by migrating **React class components to Hooks**, safely upgrading **Node.js runtimes**, and resolving high-impact issues such as **race conditions**, **memory leaks**, and unstable builds through profiling, contract testing, performance tuning, and controlled rollout strategies.

SKILLS

- **Frontend: React 16–19, TypeScript 3.x–5.x**, JavaScript ES6+, **Next.js 12–15**, React Hooks, Vite, Webpack, Storybook, reusable component architecture, SPA development, SSR/CSR patterns, route protection, lazy loading, dynamic imports, form handling, client-side validation, HTML5, CSS3, SCSS, Tailwind CSS, responsive UI, accessibility-minded UI development
- **Backend & Integration:** REST API integration, GraphQL integration, JWT authentication flows, OAuth2 flows, RBAC-aware UI, WebSockets, API versioning awareness, file upload handling, background-job aware frontend integration, OpenAPI/Swagger collaboration
- **Data & State:** Redux Toolkit, Zustand, React Query, Context API, pagination patterns, caching-aware UI behavior, local/session storage strategies, table virtualization concepts, query parameter synchronization, form state orchestration, optimistic updates, error-state modeling
- **Cloud & DevOps:** AWS collaboration, Docker, GitHub Actions, GitLab CI, Nginx environment awareness, Linux basics, frontend build pipelines, artifact packaging, release coordination, environment configuration, CI validation support
- **Observability / Quality / Security:** Jest, React Testing Library, Cypress, Playwright, unit testing, integration testing, end-to-end testing, structured logging collaboration, Sentry, OpenTelemetry basics, correlation IDs, OWASP-aware frontend practices, route protection, rate-limit aware UX handling, audit-sensitive UI flows

PROFESSIONAL EXPERIENCE

Senior Software Engineer

Soft Build, Remote (Jan2024 – Dec 2025)

- Led delivery of a multi-tenant administrative platform using **React**, **TypeScript**, and **Next.js**, reshaping unstable dashboard modules into reusable page patterns that improved consistency across tenant-specific workflows and reduced UI-related support noise by **32%**.
- Reworked large table and filtering screens that had become sluggish under heavy datasets by profiling render paths, memoizing expensive selectors, and improving cache lifecycles, which reduced visible interface freezing by **45%** for daily operations teams.
- Implemented a more maintainable component structure around forms, cards, filters, and modal flows, allowing new business features to be delivered with less duplicated UI code and helping the team shorten feature assembly effort by **29%**.
- Introduced **React Query** for async state handling in data-heavy views where manual loading flags and repetitive refetch logic had created edge-case bugs, improving request predictability and reducing stale-screen incidents by **24%**.
- Applied modern **React 18** patterns such as transition-aware updates in interaction-heavy screens, which kept urgent user actions responsive while background refreshes continued without locking the interface during large data operations.
- Stabilized real-time status widgets that were losing synchronization during temporary network issues by improving **WebSocket** reconnection behavior and buffering rules, which reduced incorrect live-state displays by **31%**.
- Solved a difficult production issue where a complex onboarding form occasionally dropped user-entered values after step changes by isolating uncontrolled state resets and replacing brittle local logic with safer centralized form handling.

- Improved frontend reliability during backend version upgrade work by coordinating payload validation changes with API teams, catching contract drift early and reducing post-release UI regressions by **22%** during rollout windows.
- Modernized older **Next.js** screen organization into cleaner feature-based folders with shared hooks, typed service layers, and stricter prop boundaries, making future onboarding easier and lowering the maintenance burden on new contributors.
- Strengthened security-sensitive screens by refining token-expiry handling, role-aware rendering, and audit-sensitive UI actions, which prevented misleading access states and reduced failed protected interactions by **18%**.
- Expanded internal search and filtering experiences around large tenant datasets by integrating scalable search response handling and clearer empty, loading, and partial-result states, helping users complete discovery tasks faster in high-volume areas.
- Improved release confidence by adding stronger **Jest**, **React Testing Library**, and end-to-end coverage on fragile flows, making regressions easier to catch before deployment and reducing hotfix frequency by **21%**.

Software Engineer

Next Apps, Remote (Nov2020 – Nov 2023)

- Built customer-facing journeys with **React** and **TypeScript** for form-heavy product areas, improving usability, validation consistency, and accessibility across flows that previously required repeated user correction and support follow-up.
- Partnered closely with product and design teams to establish a reusable frontend component library, which reduced repeated implementation work across teams and improved visual consistency for fast-moving product releases by **34%**.
- Introduced the new JSX transform during frontend modernization work, removing unnecessary imports and simplifying component files, while also creating cleaner templates that slightly improved bundle efficiency and developer ergonomics.
- Migrated shared state handling from legacy **Redux** patterns into a lighter **Zustand** approach where appropriate, reducing boilerplate and making store logic easier to reason about for hook-based feature development by **35%**.
- Improved a heavy **Webpack** pipeline that had become difficult to maintain by trimming misconfigured plugins, reorganizing build settings, and addressing source-map overhead, which shortened local build time and reduced CI instability by **27%**.
- Solved intermittent **502/504** user-facing failures that looked like frontend issues but were triggered by slow upstream behavior, then improved error messaging, retry handling, and degraded-state UX so failures became clearer and less disruptive.
- Fixed a tricky data consistency bug where rapid updates produced stale values in the interface by tightening event ordering assumptions and refreshing client reconciliation behavior around delayed asynchronous responses.
- Supported migration of older API integration patterns into cleaner service-layer abstractions, making endpoint version changes less invasive for UI code and reducing the number of frontend files touched during backend contract evolution.
- Implemented smarter caching and refetch strategies for high-traffic screens, which reduced redundant requests during repeated navigation and helped lower interface latency for frequently visited product paths by **23%**.
- Strengthened authentication-related frontend behavior by refining token validation handling, safer sign-out flows, and abuse-aware request treatment, which reduced confusing auth edge cases and improved protected-route stability.
- Improved production diagnostics by wiring clearer client-side error reporting, correlation-friendly request tracing, and better release discipline, helping the team identify issue origin faster and reduce risky production rollback scenarios by **19%**.

Software Developer

Datamart, Remote (Feb 2016 – Sept 2020)

- Built internal analytics interfaces with **React** for reporting-heavy business users, translating large backend datasets into responsive dashboards with server-side pagination, clearer filters, and more usable result navigation.
- Improved reporting performance on slow screens by aligning frontend rendering strategy with backend query optimization work, which reduced long wait perception and helped overall query-driven page response improve by **41%**.
- Modernized older **React** areas by moving class-based implementations into function-oriented component patterns as hooks became standard, which simplified lifecycle logic and made future feature updates easier to implement safely.
- Resolved a difficult cross-layer data bug that caused reporting mismatches between UI, API, and database output by standardizing parsing and display rules, restoring user trust in exported and on-screen operational numbers.

- Implemented asynchronous export status handling for long-running reports so users no longer waited on blocking browser requests, which improved transparency and reduced repeated export submissions during busy reporting periods by **26%**.
- Reduced frontend bundle weight by applying **Webpack** code splitting and removing unused dependencies that had accumulated over time, improving initial page load performance and making navigation feel lighter across major dashboards.
- Added integration-focused test coverage around business-critical flows where frequent release changes had caused regressions, giving the team a safer baseline for enhancement work and reducing repeated defect reappearance by **17%**.
- Improved CI reliability by separating fast validation from slower integration verification, which gave developers faster feedback during normal commits while preserving confidence on higher-risk release branches.
- Introduced clearer audit-oriented user feedback in update workflows so operations teams could better understand what changed and when, supporting investigation needs without making screens unnecessarily complex.
- Supported staged rollout and smoke-check practices during releases, helping catch environment-specific UI issues earlier and reducing production-facing front-end configuration surprises by **14%**.

Junior Software Developer

Berg Software, Timișoara, Romania (Sep2014 – Mar 2016)

- Built early single-page application modules with practical JavaScript and API-driven UI behavior, helping deliver more interactive business pages while learning to keep frontend interactions predictable and easy to maintain.
- Improved older web pages by adding modular client-side logic, stronger validation, and clearer form feedback, which reduced user entry mistakes and support requests by **28%** in actively used admin areas.
- Helped solve a recurring logout problem that frustrated users by tracing session expiry behavior between browser and server assumptions, then supporting a safer cookie and timeout alignment that improved login reliability.
- Created reusable UI helpers and shared page patterns that reduced duplicated implementation across internal screens, giving the team a faster path for small enhancements and more consistent behavior in repeated interactions.
- Improved page load speed by enabling compression, tightening asset delivery behavior, and removing blocking scripts that delayed initial rendering, which made older pages feel noticeably more responsive to users.
- Supported bug investigation by reproducing defects locally, reviewing logs, and delivering focused low-risk fixes under senior guidance, building a reputation for careful debugging instead of disruptive overcorrection.
- Helped maintain safer SQL-backed admin experiences by updating query logic and supporting basic indexing improvements, which kept growing data views usable as operational datasets gradually expanded.
- Contributed to release preparation through simple deployment scripts and documentation updates, helping reduce avoidable mistakes during manual release steps and making team handoffs more dependable.
- Practiced structured debugging habits by documenting reproduction paths, isolating likely causes, and validating fixes step by step, which improved fix quality and lowered the chance of repeat issues after deployment.

EDUCATION

Bachelor's Degree in Computer Science

University of Bucharest | (2010 – 2014)

- Focused on practical software engineering fundamentals that translated directly into building reliable web systems and data-driven applications.

CERTIFICATIONS

JavaScript Developer Certification — 2022

- Demonstrated strong understanding of modern **JavaScript**, asynchronous behavior, DOM interaction, modular design, debugging, and maintainable client-side development practices for interactive web applications.

Front-End Web Developer Professional Certificate — 2022

- Validated practical capability in responsive UI development, accessibility-minded implementation, reusable component thinking, browser behavior awareness, and structured frontend problem solving.